

P0153R0 `std::atomic_object_fence(mo, T&&...)`

Author: Olivier Giroux
Contact: ogiroux@nvidia.com
Author: JF Bastien
Contact: jfb@google.com
Date: 2015-11-05
Previous: <http://wg21.link/N4522>
URL: <https://github.com/jfbastien/papers/blob/master/source/P0153R0.rst>

Rationale

Fences allow programmers to express a conservative approximation to the precise pair-wise relations of operations required to be ordered in the happens-before relation. This is conservative because fences use the sequenced-before relation to select vast extents of the program into the happens-before relation.

This conservatism is commonly desired because it is difficult to reason about operations hidden behind layers of abstraction in C++ programs. An unfortunate consequence of this is that precise expression of ordering is not possible in C++ currently, which makes it easy to over-constrain the order of operations internal to synchronization primitives that comprise multiple atomic objects. This constrains the ability of implementations (compiler and hardware) to reorder, ignore, or assume the absence of operations that are not relevant or not visible.

In existing practice, the `flush` primitive of OpenMP is more expressive than the fences of C++ in at least this one sense: it can optionally restrict the ordering of operations to a developer-specified set of memory locations. This is enough to exactly express the required pair-wise ordering for short lock-free algorithms. This capability isn't only relevant to OpenMP and would be further enhanced if it was integrated with the other facets of the more modern C++ memory model.

An example use-case for this capability is a likely implementation strategy for N4392's `std::barrier` object. This algorithm makes ordered modifications on the atomic sub-objects of a larger non-atomic synchronization object, but the internal modifications need only be ordered with respect to each other, not all surrounding objects (they are ordered separately).

In one example implementation, `std::barrier` is coded as follows:

```
struct barrier {
    // Some member functions elided.
    void arrive_and_wait() {
        int const myepoch = epoch.load(memory_order_relaxed);
        int const result = arrived.fetch_add(1, memory_order_acq_rel) + 1;
        if (result == expected) {
            expected = nexpected.load(memory_order_relaxed);
            arrived.store(0, memory_order_relaxed);
            // Only need to order {expected, arrived} -> {epoch}.
            epoch.store(myepoch + 1, memory_order_release);
        }
        else
            while (epoch.load(memory_order_acquire) == myepoch)
                ;
    }
private:
    int expected;
    atomic<int> arrived, nexpected, epoch;
};
```

The release operation on the epoch atomic is likely to require the compiler to insert a fence that has an effect that goes beyond the intended constraint, which is to order only the operations on the barrier object. Since the barrier object is likely to be smaller than a cache line and the library's implementation can control its alignment using `alignas`, then it would be possible to compile this program without a fence in this location on architectures that are cache-line coherent.

To concisely express the bound on the set of memory operations whose order is constrained, we propose to accompany `std::atomic_thread_fence` with an `object` variant which takes a reference to the object(s) to be ordered by the fence.

Proposed addition

Under 29.2 Header `<atomic>` synopsis [atomics.syn]:

```
namespace std {
    // 29.8, fences
    // ...
    template<class... T>
    void atomic_object_fence(memory_order, T&&... objects) noexcept;
}
```

Under 29.8 Fences [atomics.fences], after the current `atomic_thread_fence` paragraph:

```
template<class... T> void atomic_object_fence(memory_order, T&&... objects) noexcept;
```

Effect: Equivalent to `atomic_thread_fence(order)` except that operations on objects other than those in the variadic template arguments and their sub-objects are un-sequenced with the fence. The objects operands are not accessed.

Note: The compiler may omit fences entirely depending on alignment information, may generate a dynamic test leading to a fence for under-aligned objects, or may emit the same fence an `atomic_thread_fence` would.

The `__cpp_lib_atomic_object_fence` feature test macro should be added.

Alternate wording

At the Kona meeting, the SG1 group expressed concerns about the current wording and suggested that it be reworked. The main concern was that the exclusive behavior expressed in the effect clause wasn't fully correct.

The authors seek comments on the following approach.

The current definition from 1.10 (13) is:

An evaluation A inter-thread happens before an evaluation B if

- (13.1) — A synchronizes with B, or
- (13.2) — A is dependency-ordered before B, or
- (13.3) — for some evaluation X
 - (13.3.1) — A synchronizes with X and X is sequenced before B, or
 - (13.3.2) — A is sequenced before X and X inter-thread happens before B, or
 - (13.3.3) — A inter-thread happens before X and X inter-thread happens before B.

An alternate wording could update (13.3.1) and (13.3.2) for the case where X is an object fence. In that case, these clauses apply if A's and B's modified memory location are named in the fence's objects parameters.

This could be done by either:

1. Adding two new (13.3.*) clauses.
2. Keeping all the wording updates in 29.8 and declaring an exception to 1.10 from there.

Example implementation

A trivial, yet conforming implementation may implement the new fence in terms of the existing `std::atomic_thread_fence` using the same memory order:

```
template<class... T>
void atomic_object_fence(std::memory_order order, T &&...) noexcept {
    std::atomic_thread_fence(order);
}
```

A more advanced implementation can overload this for the single-object case on architectures (or micro-architectures) that have cache coherency with a known line size, even if it is conservatively approximated:

```
#define __CACHELINE_SIZE // Secret (micro-)architectural value.
template <class T>
std::enable_if_t<std::is_standard_layout<T>::value &&
    __CACHELINE_SIZE - alignof(T) % __CACHELINE_SIZE >= sizeof(T)>
atomic_object_fence(std::memory_order, T &&object) noexcept {
    asm volatile("" : "+m"(object) : "m"(object)); // Code motion barrier.
}
```

To extend this for multiple objects, an implementation for the same architecture may emit a run-time check that the total footprint of all the objects fits in the span of a single cache line. This check may commonly be eliminated as dead code, for example when the objects are references from a common base pointer.

The above `std::barrier` example's inner-code can use the new overload as follows:

```
if (result == expected) {
    expected = nextected.load(memory_order_relaxed);
    arrived.store(0, memory_order_relaxed);
    atomic_object_fence(memory_order_release, *this);
    epoch.store(myepoch + 1, memory_order_relaxed);
}
```

It is equivalently valid to list the individual members of `barrier` instead of `*this` . Both forms are equivalent.

Less trivial implementations of `std::atomic_object_fence` can enable more optimizations for new hardware and portable program representations.

Relation to P0154R0

In [P0154R0](#) we propose to formalize the notions of false-sharing and true-sharing as perceived by the implementation in relation to the placement of objects in memory. In the expository implementation of the previous section we also showed how a cache-line coherent architecture or micro-architecture can elide fences that only bisect relations between objects that are in the same cache line, if provable at compile-time. These notions interact in a virtuous way because P0154R0’s abstraction enables reasoning about likely cache behavior that implementations can optimize for.

The example application of `std::atomic_object_fence` to the `std::barrier` object is improved by combining these notions as follows:

```
alignas(std::thread::hardware_true_sharing_size) // P0154
struct barrier {
    // Some member functions elided.
    void arrive_and_wait() {
        int const myepoch = epoch.load(memory_order_relaxed);
        int const result = arrived.fetch_add(1, memory_order_acq_rel) + 1;
        if (result == expected) {
            expected = nexpected.load(memory_order_relaxed);
            arrived.store(0, memory_order_relaxed);
            atomic_object_fence(memory_order_release, *this); // P0153
            epoch.store(myepoch + 1, memory_order_relaxed);
        }
        else
            while (epoch.load(memory_order_acquire) == myepoch)
                ;
    }
private:
    int expected;
    atomic<int> arrived, nexpected, epoch;
};
```

By aligning the barrier object to the true-sharing granularity, it is significantly more likely that the implementation will be able to elide the fence if the architecture or micro-architecture has cache-line coherency. Of course an implementation of the Standard is free to ensure this by other means, we provide this example as exposition for what developer programs might do.

Memory model example

T0	T1
0: w = 1;	4: while(!a.load(rlx));
1: x = 1;	5: objfence(acq, a, x);
2: objfence(rel, a, x);	6: assert(x);
3: a.store(1,rlx);	7: assert(w);

The semantics of fences mean that:

2 synchronizes-with 5 because [29.8¶2]:

- 2 is sequenced-before 3 ,
- 3 inter-thread happens-before 4 , and
- 4 is sequenced-before 5 .

1 happens-before 6 because [1.10¶13-14]:

- 1 is sequenced-before 2 ,
- 2 synchronizes-with 5 , and
- 5 is sequenced-before 6 .

Therefore the program is well-defined (so far) and the `assert(x)` of 6 does not fire.

However, the un-sequenced semantics of the object fence also mean that:

0 conflicts with 7 because [1.10¶23]:

- 0 is a store to w , 7 is a load of w and they are not both atomic, and
- 0 is not sequenced-before 2 and 5 is not sequenced-before 7 .

Therefore the `assert(w)` of 7 makes the program undefined due to a data-race.